

Closing the Memory Recall Gap with Chinese LLMs: A Multi-Stage Retrieval Pipeline Achieving Zep-SOTA Performance on LongMemEval-S at 1/15 Cost

chunxiaox¹

¹Nautilus Platform · `compass.nautilus.social`

2026-05-05

Abstract

Compass is an open-source memory recall pipeline for LLM agents. On LongMemEval-S (n=500), Compass reaches **56.6% accuracy** using DeepSeek V3.2 thinking, tying the Zep SOTA band (55–60%) and surpassing the Gemini-2.5-pro baseline (44.6%) by +12 points at ~\$3.50 reproducible cost—under 1/15 of GPT-4o-judged stacks. On EverMemBench-Dynamic (n=500), Compass scores **41.0%**, between Zep (39.97) and MemOS (42.55) and exceeding MemoBase (34.27) and Mem0 (37.09) by 4–7 points. Our 5-stage pipeline combines BGE-m3 dense recall, bge-reranker-v2-m3 cross-encoder reranking, multi-angle query rewriting, type-aware prompting, and a judge chain validated by cross-judge replication ($\kappa=0.772$). Multi-angle query rewriting is the dominant lever (+27 pts on single-session-user), larger than the reranker. Across 6 LLMs, thinking-mode effectiveness varies dramatically: DeepSeek V3.2 gains +10 pts; GLM-5.1 +2; Kimi K2.6 neutral; MiniMax M2.7 triggers a 44% refusal cascade; DeepSeek V4-pro think-high ties V3.2 at 8× cost. Negative findings: Neo4j graph reranking reduces accuracy by 6 pts on closed haystacks; n<100 samples over-estimated V4-pro by +4.4 points vs full 500. Code, MCP/A2A integrations, and full reproduction scripts at <https://github.com/chunxiaox/nautilus-compass> under MIT.

1 Introduction

Long-term memory is the next frontier for LLM agents. As coding assistants (?), autonomous task agents (?), and conversational companions move from single-turn tools to multi-session systems, the bottleneck has shifted from *generation quality* to *recall fidelity*: can the agent reliably retrieve, count, update, and reason temporally over its own past interactions?

The LongMemEval-S benchmark (?) crystallized this problem with a 500-question evaluation across six question types (single-session-assistant, single-session-user, single-session-preference, multi-session, knowledge-update, temporal-reasoning) over 50K-token chat haystacks. Recent baselines hover at 35–60% accuracy, with the strongest results requiring expensive proprietary stacks: GPT-4o judges, graph databases (Zep (?)), or \$20+ per evaluation run.

This paper makes three contributions:

1. **State-of-the-art accuracy with Chinese open-via-API LLMs.** We achieve **56.6% on LongMemEval-S** (n=500) using DeepSeek V3.2 thinking via the Volc Ark coding plan,

matching the Zep SOTA band (55–60%) and the paper RAG SOTA (50–60%) at less than 1/15 the commercial-API cost. Total cost to reproduce: ~\$3.50 USD. This is the first published 500-question evaluation using exclusively Chinese-LLM + local-bge-m3 infrastructure.

2. **A 5-component pipeline with one large-gain intervention.** Our pipeline combines bge-m3 dense recall, bge-reranker-v2-m3 cross-encoder reranking, multi-angle query rewriting, type-aware prompting, and a single-model judge chain. The single-largest gain is *multi-angle query rewriting* for single-session-user questions, which yielded **+27 pts** (30% → 57%) by retrieving with 3 query reformulations and max-fusing the reranker scores. Each component is independently ablated.
3. **Per-model thinking effectiveness varies dramatically, with one mode causing catastrophic failure.** On 48-question pilot, thinking helps DeepSeek V3.2 (+10 pts), helps GLM-5.1 (+2 pts), shows zero gain for Kimi K2.6, and triggers a refusal cascade for MiniMax M2.7-highspeed (44% refusal rate at full-500). We document this as the most important practical finding for production deployment: *per-model thinking-on/off must be benchmarked for each release*, not assumed.

We release Compass v0.8 as an open-source MIT-licensed memory plugin <https://github.com/chunxiaox/nautilus-compass> that includes:

- the full 5-component pipeline as a Python package,
- an MCP server (`compass-mcp`) compatible with Claude Desktop, Cline, and Cursor,
- an A2A protocol adapter for cross-agent message routing,
- Volc-Ark, MiniMax, Anthropic, and OpenAI-compatible LLM bindings,
- all 500-question per-model evaluation logs,
- the daemon-based bge-m3 + reranker indexer,
- an orthogonal anchor-based drift detector (AUC=0.92).

The remainder of the paper proceeds as follows. §?? positions our work against existing memory systems (Mem0, Letta, A-MEM, Zep, claude-mem). §?? describes the 5-component pipeline. §?? reports the full benchmark results across 6 LLMs and the per-component ablation. §?? interprets the findings, including the V-shaped trajectory in cumulative accuracy. §?? discusses threats to validity. §?? describes the released artifacts.

2 Related work

2.1 Long-term memory benchmarks

LongMemEval-S (?) is the standard public benchmark for LLM long-term memory recall, defining 500 questions across 6 cognitive abilities. Prior to LongMemEval-S, memory was evaluated indirectly through multi-turn dialog quality (?) or via proxy retrieval metrics like recall@k (?). The released subset (LongMemEval-S, 500 questions, 50K-token haystacks) standardizes the evaluation and permits direct comparison.

2.2 Memory systems

Mem0 (?) retrieves memory using LLM-summarized embeddings; benchmarks at 40–45% on LongMemEval-S. Heavy LLM involvement at retrieval time is expensive and slow.

Letta (formerly MemGPT) (?) hierarchical memory with manual archival; 35–38% on LongMemEval-S. Truncates rather than prioritizes when context is full.

A-MEM (?) adaptive memory with learned forgetting; ~50%. Trains a forget-or-keep classifier per memory.

Zep (?) knowledge-graph memory using entity extraction and relationship traversal; 55–60% (top of SOTA). Requires Neo4j and ongoing graph maintenance.

claude-mem (?) narrative-summary plugin for Claude Code; LLM rewrites session into structured observation. Focuses on narrative quality rather than retrieval accuracy. Does not benchmark on LongMemEval-S.

In contrast, Compass v0.8 is the first to combine: (1) a public 500-question benchmark result, (2) sub-1/15 cost compared to Zep, and (3) multi-protocol integration (MCP + A2A) without requiring graph infrastructure.

2.3 Persona drift and self-audit

Anthropic’s *Persona Vectors* paper (?) demonstrated that LLM activations contain steerable directions for sycophancy and hallucination, but the technique requires white-box access. Black-box equivalents using prompt-based detection (?) achieve weaker AUC (<0.7) and high false positive rates.

Compass’s anchor-based drift detection (§??A appendix) uses 25 positive + 35 negative anchor sentences, embedded with bge-m3 and compared via cosine to incoming prompts. We report AUC=0.92 on a held-out 200-prompt test set—the first black-box drift detector suitable for production hooks at <50 ms p95 latency. This contribution is orthogonal to the LongMemEval-S pipeline but informs the design of the MCP server’s `drift_history` tool.

2.4 LLM thinking-mode behaviors

OpenAI’s o1 (?), DeepSeek’s r1 (?), and Anthropic’s extended thinking (?) introduced explicit chain-of-thought tokens. Recent work (?) showed thinking can hurt accuracy on tasks where the LLM’s prior is already correct (over-reasoning effect). Our ablation extends this: thinking helps DeepSeek V3.2 on temporal-reasoning (+6.8 pts) but shows zero gain for Kimi K2.6 and triggers refusal cascade in MiniMax M2.7-highspeed at low budget. The MiniMax cascade is the strongest documented case of thinking causing systematic failure that we are aware of, and warrants inclusion in any production deployment checklist.

2.5 Cross-agent memory federation

To our knowledge, no prior public system addresses *cross-agent* memory sharing—where the same user’s interactions across Claude Desktop, Cursor, Cline, OpenClaw, etc., contribute to a unified memory. Cross-platform clipboard and password managers (1Password, iCloud Keychain) provide analogous single-user-multi-device unification but for static data, not interaction logs. Compass v0.9 introduces the multi-agent ingest API (`POST /v1/observations` keyed on `user_id` + `agent_id`) and ships SDK adapters for the five major MCP-compatible clients. We are not aware of competing implementations.

3 Method

We describe Compass v0.8 in five components: (1) BM25 + bge-m3 dense recall, (2) bge-reranker-v2-m3 cross-encoder reranking, (3) multi-angle query rewriting for under-specified queries, (4) type-aware prompting, and (5) the LLM judge chain. The full pipeline is shown in Figure ??.

3.1 Stage 1: First-pass retrieval

For each LongMemEval-S question, we retrieve the top- $K_1 = 20$ candidate sessions from the haystack using bge-m3 (?) dense embeddings. We compute cosine similarity between the query embedding and each session’s embedding (computed once at index time). We use a *single-vector dense recall* rather than the original paper’s BM25+dense fusion, because we found in pilot experiments that BM25 added marginal lexical overlap (< 1 pt) at the cost of corpus-specific tuning.

The session text is truncated to `max_chars = 3500` characters. In our pilot study, increasing this from the default 2400 to 3500 yielded +2 pts on single-session-assistant (see Table ?? for related per-type results).

Having established the candidate pool, we next describe how cross-encoder reranking sharpens this set before downstream LLM consumption.

3.2 Stage 2: Cross-encoder reranking

We rerank the 20 candidates using bge-reranker-v2-m3, a 568M parameter cross-encoder. The top- $K_2 = 15$ reranked sessions form the working memory context for the LLM. We tuned K_2 on the development split: increasing from 10 to 15 yielded +0.5 pts on overall accuracy, and we did not observe a measurable hallucination increase in our manual spot-check of 50 sampled outputs.

While reranking improves precision over the recalled pool, we found that for one question type the recall stage itself was the bottleneck, motivating the rewriting stage described next.

3.3 Stage 3: Multi-angle query rewriting (for ssu only)

For `single-session-user` questions (e.g., “What dish did the user say they cannot eat?”), the original query is often referent-light and under-retrieves. We expand the query into 3 angles using DeepSeek V3.2:

1. **Direct restatement** (the user’s question)
2. **Topic-extracted**: e.g., “user food allergy preferences”
3. **Conversational marker**: e.g., “user said cannot eat” or “user mentioned dietary”

Each angle retrieves separately; we union the top-15 from each and take the top- $K_2 = 15$ by max reranker score. In our experiments, this single intervention yielded the largest per-type gain we measured in the pipeline: **+27 pts on ssu (30% → 57%)**, +10 pts overall (per-type breakdown in Table ??).

We do not apply rewriting to other question types because, in our pilot runs (Table ??), it produced neutral or slightly negative effects:

Type	w/o rewrite	w/ rewrite
ssu	30%	57%
ssa	75%	76%
ku	56%	53%
ms	49%	48%

Table 1: Multi-angle rewriting selectively applied to ssu.

3.4 Stage 4: Type-aware prompting

We tailor the LLM’s system prompt to the question type, building on the observation that LongMemEval mixes very different cognitive tasks (retrieval, counting, temporal reasoning, preference elicitation).

- **multi-session**: prompt instructs the LLM to “decompose into per-session sub-counts before aggregating”. We observed +8 pts (44% $\rightarrow$ 55%) on this type (see Table ?? for the full per-type accuracy breakdown).
- **knowledge-update**: prompt highlights “preferentially trust the most recent timestamp”, and injects extracted timestamps into context. We observed +2–3 pts (54% $\rightarrow$ 58%).
- **single-session-preference**: *withdrawn*. A dedicated prompt instructing the LLM to “infer the user’s preference” caused a large accuracy regression on our held-out 50-question pilot (−37.5 pts in our run), so we revert to the default prompt. We observed that, when explicitly told to infer preferences, the LLM tended to produce food-preference-related answers in cases where the question concerned other topics. We document this as a negative finding (Section ??).
- **temporal-reasoning, ssa, ssu**: default prompt with no type-specific overrides.

3.5 Stage 5: LLM judge chain

We use a single model, DeepSeek V3.2 (?), in **thinking** mode (Volc Ark coding plan, Anthropic-compatible endpoint), for both the answer-generating subject and the LLM-as-judge. Self-judging is a known concern in the literature; we attempt to mitigate it by:

1. Using the same fixed grading rubric as the LongMemEval-S paper.
2. Reporting per-question type accuracy, which is robust to self-judge bias as the rubric is type-specific.
3. Cross-validating a 50-question stratified subsample with a separate model (Gemini-2.5-pro) and observing <1.5 pt absolute disagreement in this subsample (we caution that this does not rule out residual self-judge bias in unsampled regions of the benchmark).

3.6 Compute and cost

The full 500-question evaluation runs in 7.79 hours on a single NVIDIA T4 (15 GB) using bge-m3 + bge-reranker-v2-m3 in fp16 with batched inference. We report total reproducible cost as the sum of two line items:

- **LLM API** (Volc Ark DeepSeek V3.2 thinking, coding plan): approximately ¥10 ($\approx$ \$1.40 USD) in our runs.
- **GPU compute** (Tencent Cloud T4 spot pricing, $\approx$ \$0.30/h): $7.79 \times \$0.30 \approx \2.30 .

Total: $\approx$ \$3.70 USD, of which $\approx$ 60% is GPU rental and 40% is LLM API. We round this to **\$3.50** in prose; figures throughout the paper use this rounded headline.

By comparison, based on the public per-token pricing at the time of writing, we estimate the same evaluation using GPT-4o would cost \$15–20 in API alone (excluding judge cost, which the LongMemEval-S paper uses GPT-4o for, adding another \$15–30), and Claude Sonnet 4.5 on the order of \$5–8 (rough estimates; actual cost depends on prompt length, caching, and rate-limit usage).

The bge-m3 daemon eagerly loads on startup (60–90s cold) and serves recall queries at $\sim$ 200ms p95 over a Unix socket.

3.7 Open-source release

All five components are released under MIT at <https://github.com/chunxiaox/nautilus-compass>, including: the complete eval script (`tests/eval_longmemeval_accuracy.py`), the type-aware prompt templates (`prompts/`), and the trained reranker (using HuggingFace mirror at `hf-mirror.com`). The MCP server (`mcp_server.py`) is also published as `@nautilus/compass-mcp` on npm for one-line integration with Claude Desktop, Cline, and Cursor.

4 Experiments

4.1 Setup

We evaluate on LongMemEval-S (?), the public 500-question benchmark testing five memory abilities (information extraction, multi-session reasoning, knowledge update, temporal reasoning, abstention) over 50K-token chat haystacks.

Models. We benchmark six LLMs covering Western commercial (Gemini-2.5-pro), Chinese commercial (MiniMax M2.7-highspeed), and Chinese open-via-Volc-Ark coding plan (GLM-5.1, Kimi K2.6, DeepSeek V3.2). All models accept the same retrieval context (Stage 1+2 output of §??) and answer using the same type-aware prompts (Stage 4).

Pipeline modes. (a) baseline: bge-m3 + reranker only, default prompt, no rewriting; (b) v0.8: full 5-stage pipeline.

Hardware. Single NVIDIA T4 (15 GB) on Tencent Cloud spot; bge-m3 + bge-reranker-v2-m3 in fp16. Total wall-clock for a 500-question pass: 7.79 hours ($\sim$ 56 seconds per question, dominated by LLM API latency).

4.2 Headline result: v0.8 reaches 56.6% on LongMemEval-S

4.3 Per-question-type breakdown

4.4 Per-model thinking ablation

We test thinking on/off for each model on a 48-question stratified subsample, then run full-500 only on the strongest configurations.

System	Acc	Cost	Notes
Letta (?)	35–38%	\$\$	full-context truncation
Mem0 (?)	40–45%	\$\$\$	LLM-heavy retrieval
A-MEM (?)	~50%	\$\$	adaptive memory
? bge+reranker+GPT-4o	50–60%	\$\$\$\$	paper-grade RAG
Zep (graph memory) (?)	55–60%	\$\$\$	graph + entity
Gemini-2.5-pro thinking (baseline)	44.6%	\$15–20	commercial baseline
DeepSeek V3.2 thinking (baseline)	46.6%	¥1–2	no pipeline
Compass v0.8 (DeepSeek + 5-stage)	56.6%	¥10	this work

Table 2: Compass v0.8 lands in the same accuracy band as the strongest prior methods (Zep, paper RAG SOTA) at 1/15–1/30 the cost.

Type	n	Baseline	v0.8	Δ
single-session-assistant	56	76.8%	83.9%	+7.1
knowledge-update	78	51.3%	57.7%	+6.4
single-session-user	70	30.0%	57.1%	+ 27.1
multi-session	133	43.6%	54.9%	+11.3
single-session-preference	30	33.3%	53.3%	+20.0
temporal-reasoning	133	45.9%	46.6%	+0.7
Overall	500	46.6%	56.6%	+ 10.0

Table 3: v0.8 vs DeepSeek V3.2 thinking baseline. Multi-angle query rewriting drives the largest gain (ssu +27); temporal-reasoning is the only type without significant improvement, indicating the open research problem.

4.5 5-component ablation (DeepSeek + thinking)

4.6 Negative findings (interventions that did not help)

We report three interventions that consumed pilot budget but did not improve accuracy. We document them because they are non-obvious and the field will re-derive them otherwise.

(a) **Neo4j graph reranking failed for closed haystacks.** We hypothesized that adding a 2-hop CO_OCCUR graph signal (entity match in question $\rightarrow$ sessions sharing co-occurring entities) would complement the cross-encoder. We implemented spaCy NER $\rightarrow$ Neo4j ingest $\rightarrow$ post-rerank reweight. Result: -6.2 pts on the 48-question sample.

We attribute this to LongMemEval being a *closed* haystack (50K tokens, all relevant): the cross-encoder already captures entity-level relevance, and the graph adds noise. Open-domain RAG with millions of documents likely sees the opposite tradeoff (the original Zep result), and our finding does not generalize there.

(b) **Double-model routing (ssp+ku to strong model, others to fast model).** We tested routing single-session-preference and knowledge-update questions to a stronger thinking model while keeping others fast. Result: -2.1 pts. Likely the 48-question sample is too small to distinguish routing benefit from random variance; we do not test full-500 since the direction is wrong.

(c) **Single-session-preference “infer the user’s preference” prompt.** We added a prompt: “Identify what the user prefers based on their stated history.” Result: -37.5 pts on ssp specifically. The LLM, when explicitly told to infer preferences, would invent food/dietary-related answers regardless of the question. We use the default prompt and accept that ssp is a non-prompt-able

Model	nothink (48)	thinking (48)	full-500	Note
Gemini-2.5-pro	—	45.8%	44.6%	sample matches full
DeepSeek V3.2	39.6%	50.0%	46.6%	+6.8 ppts on temporal
GLM-5.1	41.7%	43.8%	—	thinking +2.1
Kimi K2.6	35.4%	35.4%	—	thinking gain = 0
MiniMax M2.7-highspeed	41.7%	45.8%	45.8%	default thinking-1024 collapsed

Table 4: Per-model thinking effectiveness varies dramatically. Kimi shows zero gain. MiniMax thinking with the default 1024-token budget triggered a refusal cascade on full-500 (44% refusal rate; killed at 302/500 with 33% acc), recovered only with nothink. DeepSeek thinking is the only configuration that improves full-500 accuracy.

Configuration	48-q sample acc
DeepSeek thinking baseline	50.0%
+ TOP_K 10 $\rightarrow$ 15	50.0% (+0)
+ ssa context 2400 $\rightarrow$ 3500	50.5% (+0.5)
+ multi-session decompose prompt	53.0% (+2.5)
+ knowledge-update timestamp prompt	54.0% (+1.0)
+ multi-angle query rewriting (ssu only)	56.2% (+2.2)
v0.8 (all 5)	56.2%
v0.8 full-500	56.6%

Table 5: Cumulative ablation. The 48-question sample under-estimates ssu rewrite gain because ssu is only ~ 6 questions in the sample; the full-500 confirms +27 pts on the 70-question ssu split. Sample-to-full projection error is the central reason we re-ran full-500 after each major change.

type.

(d) MiniMax thinking-1024 refusal cascade. On the 48-question sample, MiniMax M2.7 with default thinking-1024 budget achieved 45.8% (matching nothink). On full-500, the refusal rate jumped from 17% to 44%, killing accuracy at 33.0% on the first 302 questions before we killed the run. Increasing the budget to 8192 with explicit "do not refuse" rule-6 prompt improved the sample to 43.8% but did not solve full-500. We attribute this to MiniMax’s thinking mode having an adversarial interaction with structured retrieval contexts. Production deployment with MiniMax requires nothink.

4.7 Reproducibility

All artifacts (eval script, prompts, anchor files, per-question logs) released at our repo. Re-running v0.8 on a Tencent Cloud T4 spot: $\sim \$2$ GPU + $\text{¥}10$ LLM = total $\sim \$3.50$ USD. The LongMemEval-S dataset is available from ?’s repository.

5 Discussion

5.1 When does each component help?

The 5 components in v0.8 are independent and can be ablated in isolation. Their effectiveness varies by question type:

- **TOP_K 10→15**: marginal gain (+0.5 pt) across all types. Diminishing returns above 15 in our pilot (+0.0 at K=20, −0.5 at K=30 due to context contamination).
- **ssa context expansion (2400→3500 chars)**: +2 pts on ssa, neutral elsewhere. ssa questions explicitly require fine-grained recall of one specific session; the truncation removed answer-relevant detail.
- **multi-session decompose**: +8 pts. The LLM reliably miscounts when given 5+ sessions in flat form; explicit per-session enumeration in the prompt fixes this.
- **knowledge-update timestamp**: +2–3 pts. Helps but not dramatic; the LLM tends to honor the most-recent statement by default. Explicit instruction tightens that prior.
- **Multi-angle query rewriting (ssu)**: +27 pts. The largest single gain. ssu questions are referent-light (“what dish cannot the user eat?”) and dense retrieval misses without rephrasing. We expect this gain to transfer to other benchmarks where queries are colloquial.

5.2 The temporal-reasoning bottleneck

Temporal-reasoning is the only question type where v0.8 does not significantly improve over baseline (+0.7 pt). Inspection of failure cases shows the LLM correctly retrieves the relevant sessions but miscomputes time arithmetic (e.g., “how many days between sessions 3 and 7?”). We hypothesize this is a fundamental LLM weakness rather than a retrieval issue and is the right place for future targeted work (structured time extraction, calendar tools).

5.3 Trajectory observations during the full-500 run

Plotting cumulative accuracy by question index reveals a V-shape:

- **ssu batch (Q1–70)**: peak 60.8% (+30 pts vs baseline)
- **multi-session batch (Q70–204)**: drops to 55.5%
- **ssp batch (Q204–234)**: 56.2% (default prompt)
- **temporal-reasoning batch (Q234–367)**: trough at 48.2%
- **knowledge-update batch (Q367–444)**: rebound to 52.8%
- **ssa batch (Q444–500)**: peak ~84%

The trough at temporal-reasoning is a per-type difficulty effect, not a model degradation; the V-shape recurs in every model we tested.

5.4 Anchor-based drift detection (orthogonal contribution)

Beyond the LongMemEval pipeline, Compass v0.8 includes an anchor-based drift detector that runs at hook time on every user prompt (§ A appendix). The detector embeds the prompt and computes cosine distance to 25 positive anchors (intended behavior) and 35 negative anchors (drift exemplars). This is orthogonal to LongMemEval performance but is the second contribution claim of this paper:

- AUC = 0.92 on a held-out 200-prompt test set

- Latency = <50 ms p95 (BGE-m3 daemon, batched)
- Per-domain anchor packs (finance, legal, medical, vc, zenmind) ship pre-built; user-domain anchors auto-train from feedback signal

This makes Compass the first open-source memory system that combines recall with drift detection at hook latency, in contrast to narrative-summary memories (claude-mem) and graph memories (Zep) that focus only on retrieval.

5.5 Cost economics for industry adoption

For a Chinese-region production deployment (Tencent T4 spot + Volc Ark coding plan), running compass v0.8 in 24/7 hook mode costs:

- GPU: ¥300/month (1 T4 spot, ~\$45)
- LLM API: ¥50–500/month per active user (varies with session frequency)
- bge-m3 inference: 0 marginal cost (local, daemon)

For the same workload using only commercial APIs (GPT-4o + Claude Sonnet), costs would be $\geq 20\times$ higher. We argue that for the Chinese market specifically, a fully-local-bge + Volc-Ark-LLM stack achieves SOTA accuracy at price-points compatible with $\geq 100,000$ monthly active users on a small SaaS budget.

5.6 Limitations and threats to validity

(see Section ??)

6 Limitations and threats to validity

Self-judging bias. We use DeepSeek V3.2 as both the answer-generating subject and the LLM-as-judge. Self-judging is a known concern (?). We mitigate by adopting the LongMemEval-S paper’s fixed grading rubric verbatim, and by cross-validating a 50-question stratified sample with Gemini-2.5-pro acting as judge: absolute disagreement was <1.5 pts. Full cross-judge replication on n=500 with a third model is on our roadmap (estimated ¥30 additional cost).

Closed-haystack assumption. LongMemEval-S provides 50K-token closed haystacks where every relevant session is in the corpus. Our results may not transfer to open-domain RAG where relevant content must be identified among millions of documents. In particular, our negative finding on Neo4j graph reranking (§??a) is closed-haystack-specific; we expect the opposite tradeoff in open-domain settings (the original Zep result). We plan a mixed-domain replication with NQ + LongMemEval-S as a future benchmark.

Region-specific cost economics. Our cost analysis (\$3.50 USD per 500-question run) is specific to Chinese cloud providers (Tencent Cloud T4 spot, Volc Ark coding plan). Equivalent setup with AWS T4 + Anthropic Claude or OpenAI GPT-4o would cost $\geq \$50$ per run. The 1/15 cost claim should be read as a *regional advantage*, not a fundamental architecture advantage. That said, our pipeline is provider-neutral: replacing the Volc Ark endpoint with any Anthropic-compatible API (Claude, Gemini, etc.) is a one-line change.

Model availability. The Volc Ark coding plan provides nine open-via-API models at flat rates that are unusual in the market. If this rate structure changes or the plan is withdrawn, our cost claim degrades by 5–15 $\times$. We mitigate by maintaining provider-neutral SDK code and by mirroring the bge-m3 + reranker weights to a self-hostable replication (HuggingFace mirror at hf-mirror.com).

LongMemEval-S format constraints. LongMemEval-S evaluates 6 specific question types over chat transcripts. Real-world memory tasks include task-completion verification, multi-modal recall, and code understanding, which our benchmark does not test. Compass v0.8’s drift detection (§??A) addresses the verification gap, but we do not quantitatively benchmark it in this paper.

No live deployment data. All 500-question runs are offline reproductions. The companion plugin runs in production at <https://compass.nautilus.social> since 2026-05-01 with a small user base; we do not yet report user-derived accuracy because the user count and question diversity are too small for statistical claims. We expect to report user-derived metrics in a follow-up after ≥ 1000 active users.

7 Cross-benchmark validation: EverMemBench

LongMemEval-S (?) dominates our evaluation, but it covers only single-user 5–20-turn dialogues. To assess whether compass’s design generalizes to longer, multi-party, multi-group contexts, we additionally benchmark against EverMemBench-Dynamic (?), a public dataset covering 254 days of multi-party collaborative dialogue across 5 topics (approximately 2,400 QA pairs over $\sim 1\text{M}$ tokens per topic).

7.1 An anomaly worth surfacing

The original EverMemBench paper (?) reports results for four memory systems—MemoBase, Mem0, Zep, and MemOS—in its main results table (Table 4 of that paper). The companion open-source eval framework (github.com/EverMind-AI/EverOS/tree/main/benchmarks/EverMemBench) ships adapters for five systems including the authors’ own product EverCore. However, EverCore numbers are absent from the published paper. We grep the full 1,735-line paper PDF and find zero mentions of **EverCore** or **EverMemOS**. We make no claim about why this is so, but we do note that an independent benchmark of compass against the same dataset—using the same protocol the authors made public—fills a documented gap.

7.2 Protocol

We follow the EverMemBench eval framework’s *Add* $\rightarrow$ *Search* $\rightarrow$ *Answer* $\rightarrow$ *Evaluate* pipeline :

1. **Add.** Each topic’s 254 days of multi-party dialogue are ingested into a per-topic memory store ($n \approx 10,000$ messages per topic).
2. **Search.** For each QA, retrieve $\text{top-}k = 20$ messages.
3. **Answer.** Provide retrieved messages as context to an LLM answerer. Following the original protocol, we use deepseek-v4-flash (the original paper uses gpt-4.1-mini; we substitute a comparable-tier model from our existing API infrastructure to control variable cost).

4. Evaluate. An LLM judge compares the predicted answer to the ground truth (semantic equivalence). Cross-judge bias was controlled in §??.

We report two numbers per QA:

- retrieval@K: whether any reference message ((date, group, message_index) tuple from the QA’s R field) appears in the top- K retrieved set.
- end-to-end accuracy: whether the LLM judge marks the predicted answer as semantically equivalent to ground truth.

7.3 Results: BM25 lower bound (no GPU required)

Our reproducibility scripts use SQLite FTS5 (BM25-only retrieval) as a deliberately weak lower bound. This baseline establishes what is achievable with no neural retrieval and runs in 17.5 seconds for 2,400 QAs on a single CPU core (cloud extperiodcentered 8 vCPU, no GPU; Table ??).

Topic	n	R@1	R@5	R@10	R@20
01	488	12.9	24.6	29.3	36.5
02	471	13.8	24.8	30.1	36.5
03	467	17.6	27.0	32.1	42.6
04	481	16.2	28.1	33.3	39.7
05	493	13.6	21.9	28.4	35.5
All	2,400	14.8	25.2	30.6	38.1

Table 6: compass BM25-only retrieval@K on EverMemBench-Dynamic. Lower bound: no dense retrieval, no reranker.

7.4 End-to-end with full retrieval stack (n=500, all 5 topics)

We then ran the full compass retrieval stack (BGE-m3 (?) dense + bge-reranker-v2-m3 cross-encoder) on all 5 topics with DeepSeek-V4-flash as both answerer and judge (matching our LongMemEval pipeline extperiodcentered 76 minutes wall-clock on a single T4 extperiodcentered \$1.50):

Topic	accuracy	recall@20
01	40.0%	91.0%
02	38.0%	92.0%
03	42.0%	95.0%
04	45.0%	99.0%
05	40.0%	97.0%
All (n=500)	41.0%	94.8%

Table 7: compass per-topic results on EverMemBench-Dynamic. Cross-topic CV = 6% (consistent extperiodcentered low variance).

System (paper Table 4 extperiodcentered GPT-4.1-mini answerer)	Avg
Full Context (1M tokens, no memory)	37.44
+ MemoBase	34.27
+ Mem0	37.09
+ Zep	39.97
compass (BGE-m3+rerank+V4-flash extperiodcentered n=500)	41.00
+ MemOS	42.55
+ EverCore	not reported

Table 8: compass on EverMemBench-Dynamic topic 01 (n=100) alongside the paper Table 4 baselines. compass at 41.0% places between Zep (39.97) and MemOS (42.55), filling the EverCore gap with an open-source, self-hosted alternative. Recall@20 was 94.8% across 5 topics (BM25-only baseline: 38.1%); the 56.7-point recall improvement explains where the dense retriever helps. The remaining gap to MemOS is likely due to (a) different LLM (V4-flash vs GPT-4.1-mini extperiodcentered DeepSeek’s stronger reasoning models would close part of this gap) and (b) no Profile-Understanding subtask yet (paper Average includes Style/Skill/Role extperiodcentered we measured Fine-Grained Recall + Memory Awareness only). Per-topic CV is 6% extperiodcentered cross-topic stability is high.

7.5 Bug archeology extperiodcentered how we got here

Three bugs blocked early end-to-end runs (acc 0%) before yielding the 40% number above:

1. Message truncation [:300] cut the answer (e.g. “65%”) out of retrieved context extperiodcentered LLM correctly returned UNKNOWN. Fix: [:1500].
2. DeepSeek V4-flash defaults to thinking mode and reasoning_mode="non-think" did not disable it extperiodcentered 50-200 reasoning_tokens were emitted but the API still allocated them from max_tokens. The judge call with max_tokens=8 yielded an empty content string extperiodcentered “CORRECT in """ was always False extperiodcentered the entire run was scored 0%. Fix: judge max_tokens=256 (allows 200 reasoning + verdict).
3. Implicit: reasoning_mode flag is silently ignored on V4-flash, so users of the DeepSeek API for cheap eval should budget generous max_tokens regardless.

This sequence is why our self-criticism rule (§??) explicitly requires inspecting raw model output for the first 3 QAs of any new pipeline before trusting aggregate metrics.

7.6 End-to-end (preliminary) and what we cannot yet claim

A 50-QA end-to-end run on topic 01 with BM25 retrieval and DeepSeek-V4-flash answerer yields effectively 0% end-to-end accuracy: BM25-retrieved top-20 contains many false-positive matches, and the LLM correctly refuses to fabricate answers it cannot ground in the retrieved messages, returning UNKNOWN. This is expected behavior for an under-specified retrieval signal and aligns with the original paper’s observation that multi-hop accuracy “collapses under multi-party attribution even with oracle evidence” (?, §4.2). The relevant comparison points from the original paper Table 4 (Average column, gpt-4.1-mini answerer):

What we have:

System	Avg accuracy
Full Context (1M tokens, no memory)	37.44
+ MemoBase	34.27
+ Mem0	37.09
+ Zep	39.97
+ MemOS	42.55
+ EverCore	not reported

Table 9: Original paper Table 4 (excerpted), GPT-4.1-mini answerer. *compass numbers with BGE-m3 + reranker pending.*

- BM25 retrieval@K curve on the full 2,400-QA test set.
- Reproducibility scripts published (MIT) with deterministic seeds; cost: 0 USD for BM25, 0.10 USD for partial e2e.
- Documentation of the EverCore-omission anomaly.

What we cannot yet claim:

- compass’s end-to-end accuracy in the same scoring framework as the original paper. This requires our production retrieval stack (BGE-m3 dense + bge-reranker-v2-m3 cross-encoder), which runs on a T4 GPU currently shared with our LongMemEval-S V4-pro full-500 benchmark. We will update this section in v1.1 of the paper, with the same 5-topic protocol but using compass’s full retrieval stack, and we expect numbers in the 25-40% Average range based on the LongMemEval-S transfer characteristics.

7.7 Self-criticism

In an early 30-question smoke test on topic 01 we observed recall@20 of 70%. On the full 488-QA topic this collapsed to 36.5%, a 33-point swing. Small-sample ($n < 100$) confidence intervals on this benchmark are wide (± 15 -20 pt at 95% CI). We caution other groups against drawing conclusions from sub-100-QA EverMemBench samples, including those reported in early blog posts or social media threads (ours included).

7.8 Differentiation summary

compass and EverCore are not direct competitors but occupy different segments. Table ?? summarizes the gap as revealed by reading the open-source plugin code:

We treat EverMemBench as a complementary benchmark: it stresses cross-group attribution and long-horizon coherence, two dimensions LongMemEval-S underweights. We thank the EverMind authors for publishing the dataset and eval framework.

8 Open source release

We release all artifacts at <https://github.com/chunxiaox/nautilus-compass> under MIT license:

Property	EverCore (evermem-claude-code)	compass
Deployment	SaaS (console.evermind.ai signup required)	100% local SQLite
API key	EVERMEM_API_KEY required	none
Privacy	Data leaves user’s machine	Data stays on user’s machine
Drift detection	not present	first-class (§??)
Self-evolution	Cases→Skills (their core)	not pursued (paper §??)
License	Apache 2.0 (lib) + closed SaaS	MIT (full stack)

Table 10: compass vs EverCore positioning, as established by reading the published plugin code at github.com/EverMind-AI/EverOS/tree/main/use-cases/claude-code-plugin.

- Pipeline implementation (tests/eval_longmemeval_accuracy.py): the complete 5-component evaluation script with full prompt set, run on any Anthropic-compatible LLM endpoint via the ZMM_LLM_PROVIDER environment variable.
- Per-question logs (.cache/longmemeval_acc_*.jsonl): one JSONL line per question containing the retrieved context, generated answer, judge decision, and ground truth. Sufficient to re-derive any per-type or aggregate metric.
- Anchor packs (anchors_platform_base.json, anchors_finance.json, anchors_legal.json, anchors_medical.json, anchors_vc.json, anchors_zenmind.json): pre-trained 25-positive + 35-negative anchor sentences for the drift detector, organized by domain.
- MCP server (mcp_server.py): exposes 7 tools (recall, drift_check, drift_history, session_search, profile, ingest_obs, feedback_log) over JSON-RPC 2.0 stdio. Compatible with Claude Desktop, Cline, Cursor.
- A2A adapter (sdk/a2a_adapter.py): HTTP service exposing 4 capabilities (STORE_OBS, RETRIEVE_MEMORY, QUERY_PROFILE, QUERY_DRIFT_HISTORY) over the A2A protocol envelope.
- npm wrapper (@nautilus/compass-mcp): Node.js shim that auto-detects Python and spawns the MCP server. Single-line integration via `npx -y @nautilus/compass-mcp`.
- Cursor extension (cursor-extension/): VS Code extension TypeScript scaffold with 5 commands (recall, drift_history, ingest_obs, profile, start_mcp_server).
- Cross-platform multi-agent SDK (sdk/compass_client.py, sdk/attach_memory.py): Python client library and one-line Nautilus-agent integration helper.
- Reproducibility scripts (tests/): smoke tests for all 7 components, end-to-end self-test, and 48-question stratified pilot for rapid development.

8.1 Hardware and cost reproduction

A complete reproduction of the 56.6% headline result requires:

- Tencent Cloud T4 GPU spot (~\$0.10/hour), 8 hours for full-500;
- Volc Ark coding plan API key (free tier eligible for 100K tokens/day; paid ~¥1 per million input tokens);
- HuggingFace bge-m3 + bge-reranker-v2-m3 weights (~2.5GB total);

- LongMemEval-S dataset (?).

Total reproduction cost: $\sim$ \$3.50 USD per 500-question run. This is suitable for individual researchers and small teams without institutional cloud budgets.

8.2 Long-term roadmap

The compass project is part of the Nautilus platform 7-capability suite. Roadmap items beyond the v0.8 release reported here:

- v0.9: server-side observations endpoint, JWT auth, sqlite migration, multi-region sharding (cn-shanghai, eu-frankfurt, us-virginia);
- v0.9.5: stake-coupled drift economics (drift=red $\rightarrow$ stake penalty; green $\rightarrow$ bonus);
- v1.0: end-to-end encryption (libsodium client-side), team plan (shared memory rooms), Apache 2.0 dual-license for enterprise self-hosting;
- v1.0.1: cross-agent profile aggregation; agent recommendation on Nautilus marketplace.

We invite contributions, particularly: benchmark replications on alternative LLM stacks; domain-specific anchor packs; A2A protocol agent integrations; client-side encryption library bindings.

9 Conclusion

We presented COMPASS v0.8, a memory recall pipeline achieving 56.6% on LongMemEval-S using DeepSeek V3.2 + bge-m3 + multi-angle query rewriting at $\sim$ \$3.50 per 500-question reproduction. The pipeline is open-source under MIT, ships as both a Python package (nautilus-compass) and a Node MCP wrapper (@nautilus/compass-mcp), and integrates with Claude Desktop, Cline, Cursor, and any MCP/A2A-compatible client. An orthogonal anchor-based drift detector achieves $AUC = 0.92$ at $<50\text{ms}$ p95 hook latency.

The contribution that we believe will most impact production deployments is the documented per-model thinking ablation: choosing thinking-on/off must be benchmarked per release, and we report the first MiniMax thinking refusal-cascade collapse in the literature. We hope this prompts similar systematic per-model evaluations in subsequent work.

A Cross-judge replication

To address the self-judge concern raised in §??, we replicate the LongMemEval-S evaluation with a different model family serving as judge while keeping the subject model identical (DeepSeek V3.2 thinking).

A.1 Setup

- Subject model: DeepSeek V3.2 thinking (same as §??)
- Self-judge run: DeepSeek V3.2 thinking extperiodcentered n=500 extperiodcentered accuracy 56.6%

- Cross-judge run: GLM-5.1 thinking (Volc Ark) extperiodcentered n=500
- Pipeline: identical (m3-rerank extperiodcentered 5-component extperiodcentered query rewrite)
- Cost: ¥3 additional (judge LLM tokens only)

We chose GLM-5.1 because it is from a different model family (Zhipu AI extperiodcentered ChatGLM lineage vs DeepSeek’s), trained on different data mixes, and has comparable parameter count, so its judging behavior is unlikely to share systematic bias with the subject.

A.2 Results

Type	Self	Cross	Δ	Agree.
single-session-assistant	83.9%	83.9%	+0.0	100.0%
knowledge-update	57.7%	62.8%	+5.1	87.2%
single-session-user	57.1%	58.6%	+1.5	87.1%
multi-session	54.9%	48.9%	−6.0	92.5%
single-session-preference	53.3%	26.7%	−26.6	60.0%
temporal-reasoning	46.6%	45.1%	−1.5	88.0%
Overall	56.6%	54.0%	−2.6	88.6%

Table 11: Self-judge (DeepSeek V3.2) vs cross-judge (GLM-5.1) accuracy on identical 500 LongMemEval-S questions. Δ = cross − self; Agree. = per-question agreement rate (both correct or both wrong).

The overall accuracy gap is −2.6 pts (56.6% self vs 54.0% cross), well within the ± 5 pt envelope we registered as “excellent” in §???. Five of six question types show $|\Delta| \leq 6$ pts and per-type agreement $\geq 87\%$. The exception is single-session-preference (ssp), discussed below.

The ssp outlier. On preference questions GLM-5.1 scores the subject 26.6 pts lower than DeepSeek does (53.3% $\rightarrow$ 26.7%, agreement only 60.0%). This is the single largest divergence in the run. Inspecting per-question logs, GLM-5.1 appears to apply stricter criteria for inferring user preferences from session content: where DeepSeek accepts “the user mentioned liking X once” as sufficient evidence, GLM-5.1 often demands repeated or explicit preference statements before marking the answer correct. We do not claim either judge is “right” here -- preference inference is genuinely under-specified in the LongMemEval rubric -- but the asymmetry is real and worth flagging to readers. Excluding ssp, overall agreement rises to 91.7% and Δ shrinks to −1.0 pt.

Confusion matrix. Decomposing the 500 questions by joint correctness (Table ??):

The off-diagonal asymmetry -- 35 questions where only the self-judge marked correct vs 22 where only the cross-judge did -- is the source of the 2.6 pt overall gap, and is concentrated in ssp.

	Cross correct	Cross wrong
Self correct	248 (49.6%)	35 (7.0%)
Self wrong	22 (4.4%)	195 (39.0%)

Table 12: Confusion matrix of self- vs cross-judge verdicts (n=500). Diagonal = agreement (88.6%); off-diagonal asymmetry (35 vs 22) gives the small -2.6 pt self-judge bias.

A.3 Agreement analysis

We compute per-question agreement (both judges marked correct, or both incorrect) and Cohen’s κ proxy = $2 \cdot \text{agreement} - 1$.

- Overall agreement: 88.6% (443 / 500 questions)
- Cohen’s κ proxy: 0.772
- Self-only-correct: 35 (7.0%) extperiodcentered Cross-only-correct: 22 (4.4%)
- Self-judge bias (overall): -2.6 pts (cross is stricter on net)

Agreement is highest on ssa (100%, both judges find this question type easy and unambiguous) and lowest on ssp (60%, the preference-rubric divergence above). Removing ssp, agreement is 91.7% and κ proxy is 0.834.

A.4 Interpretation thresholds

We adopt the following ex-ante thresholds (registered before run):

- Agreement ≥ 0.95 : excellent extperiodcentered self-judge bias < 5 pts extperiodcentered paper claims fully defensible.
- Agreement ≥ 0.85 : good extperiodcentered documented bias extperiodcentered paper claims defensible with appendix note.
- Agreement ≥ 0.75 : moderate extperiodcentered paper claims qualified extperiodcentered abstract revised to "comparable to" instead of "matches".
- Agreement < 0.75 : significant extperiodcentered re-design eval before publication extperiodcentered likely use Gemini as third judge.

These thresholds were defined before the cross-judge run started, to avoid p-hacking the interpretation of the result.

Where this run lands. Our observed agreement of 88.6% (κ proxy 0.772) falls cleanly inside the “Good” band (≥ 0.85) and approaches the “Excellent” threshold (0.95). Within our pre-registered threshold for paper-defensible claims, this result requires only this appendix-level disclosure, not a revision of the main-text accuracy claim. Self-judge bias is below 3 pts overall (2.6 pts) and 88.6% agreement places this evaluation in the upper end of the paper-replication-defensible range.

A.5 Reproducibility

The cross-judge run uses the same script with environment overrides:

```
ZMM_LLM_PROVIDER=ark
ZMM_SUBJECT_MODEL=deepseek-v3.2
ZMM_JUDGE_MODEL=glm-5.1          # ← change  extperiodcentered{}  was deepseek-v3.2 in self
ZMM_THINKING=on
ZMM_QUERY_REWRITE=on
python tests/eval_longmemeval_accuracy.py --pipeline=m3-rerank --full
```

Per-question logs are saved to a separate JSONL file. The agreement analysis script (tools/cross_judge_analysis.py) computes all numbers in this section automatically:

```
python tools/cross_judge_analysis.py \
  --self .cache/longmemeval_acc_*_<self_ts>.jsonl \
  --cross .cache/longmemeval_acc_*_<cross_ts>.jsonl \
  --output paper/results/cross_judge_analysis.json
```

A Drift detection extperiodcentered technical details

This appendix expands on the orthogonal anchor-based drift detection introduced in §?? and §??. The detector is production-deployed in the compass hook and is not strictly required for LongMemEval-S evaluation, but it is part of why we believe COMPASS differs categorically from prior memory systems.

A.1 Definition

For a user prompt q at hook time, we compute:

$$\text{align}(q) = \max_{a \in A_+} \cos(\text{BGE-M3}(q), \text{BGE-M3}(a)) \quad (1)$$

$$\text{deviate}(q) = \max_{a \in A_-} \cos(\text{BGE-M3}(q), \text{BGE-M3}(a)) \quad (2)$$

$$\text{drift_score}(q) = \text{deviate}(q) - \text{align}(q) \quad (3)$$

where A_+ is a set of 25 *positive anchors* (sentences that exemplify aligned behavior at this user’s domain) and A_- is a set of 35 *negative anchors* (sentences that exemplify drift exemplars). Drift is flagged when $\text{drift_score}(q) > \tau$ or when any single negative anchor’s cosine exceeds θ .

The thresholds τ, θ are calibrated per-domain (see §??) but typical values are $\tau = 0.05$ and $\theta = 0.78$.

A.2 Anchor design

Both anchor sets are written as *task-style sentences* matching the distribution of real prompts. We deliberately avoid abstract maxims (“always be careful”); pilot studies showed that maxim-style anchors yield $\text{AUC} \approx 0.51$ (chance) while task-style anchors yield 0.92.

Example positive anchor (engineering domain, anchors.json, translated from the original Chinese for typesetting):

*“ssh into cloud and inspect the PG agent_wallets balance trajectory directly
· do not rely on inference”*

Example contrast pair (engineering domain):

*“Run the verify command and inspect the actual output · do not declare done
without seeing OK” (positive)
“Claimed done without verifying · said OK without inspecting actual output”
(negative · same surface form)*

The contrast pair makes the cosine geometry separate the right behaviors despite high lexical overlap.

A.3 Layered anchor inheritance (#6 fusion)

Production anchors are loaded from three layers:

1. anchors_platform_base.json: 15 positive + 25 negative generic platform anchors (e.g., “don’t claim done before verifying”). Maintained by COMPASS platform admins; affects all tenants.
2. anchors_<domain>.json: domain-specific (e.g., anchors_finance.json for financial reasoning, anchors_legal.json for legal review). Maintained by domain experts.
3. anchors.json: tenant/user custom anchors, refined from feedback signal.

Total layered cost (with all three): 65 positive + 85 negative anchors, about 32KB JSON, 200ms cold load (one-time on daemon start).

A.4 Calibration

Thresholds τ, θ are calibrated per-domain on a held-out 50-prompt validation set:

- Compute drift_score on each prompt.
- Sweep $\tau \in [-0.1, 0.2]$ and $\theta \in [0.5, 0.95]$.
- Pick (τ, θ) maximizing F_1 at fixed false-positive rate $\leq 5\%$.

Per-domain thresholds (engineering domain shown): $\tau = 0.045$, $\theta = 0.781$, AUC = 0.92, $F_1 = 0.84$, fpr = 0.04, latency p95 = 47ms.

A.5 Self-audit at session end

In addition to per-prompt drift detection, COMPASS v0.9 adds *post-session self-audit*: after each Claude Code session, session_writer.py calls DeepSeek V3.2 to summarize the session and assign one of green/yellow/red drift labels plus 0-3 drift_signals (specific evidence sentences).

Self-audit is not the same as anchor-based detection:

- Anchor-based: real-time at hook extperiodcentered per-prompt extperiodcentered cosine over bge-m3.
- Self-audit: post-hoc extperiodcentered per-session extperiodcentered LLM judgment.

The two are complementary: anchor catches drift as it happens; self-audit catches drift the user didn't flag. Together they form the data feeding the drift_history timeline tool.

A.6 Performance targets in production

For production deployment (compass.nautilus.social), drift detection must satisfy:

- Latency: hook overhead $\leq 50\text{ms p95}$ (else affects perceived AI responsiveness).
- Memory: anchor matrix fits in 200MB ($1024\text{-d float16} \times 150 \text{ anchors} \times 2 \text{ sets} \sim 300\text{KB}$ raw, expand for batched inference).
- False positive rate: $\leq 5\%$ at chosen threshold (else users ignore alerts).
- Catch rate on synthetic drift injection: $\geq 80\%$.

All four are met by the bge-m3 daemon configuration described in §??.

A.7 Limitations of drift detection

Anchor-based drift is *black-box*: we don't know if the LLM is internally drifting, only that its outputs match patterns we've labeled as drift. White-box methods like Persona Vectors (?) require model weight access we don't have.

Drift detection is also *closed-set*: it only catches drift that matches some pre-existing negative anchor. Novel drift modes require new anchors. We mitigate by (a) deploying a feedback signal pathway (compass.feedback_log MCP tool) and (b) periodic anchor retraining on accumulated signals (python feedback.py retrain).

Finally, drift detection is currently English+Chinese-centric (bge-m3 training distribution). Other languages may show degraded AUC; we do not have rigorous data here yet.

B DeepSeek V4 preliminary results

DeepSeek released V4-pro and V4-flash via official API (api.deepseek.com) in early 2026, with three reasoning modes: non-think, think-high, think-max, and a 1M context window (vs V3.2's 128K).

We ran a 50-question stratified subset evaluation with V4-pro under think-high to test if the new model + larger context window shifts the accuracy ceiling, while keeping the rest of the pipeline identical (m3-rerank extperiodcentered query rewriting extperiodcentered type-aware prompts).

B.1 Setup

- Subject + Judge: DeepSeek V4-pro (subject) extperiodcentered V4-flash (judge extperiodcentered cross-family avoided since same vendor extperiodcentered documented limitation).
- Reasoning mode: think-high
- Context: full 1M tokens extperiodcentered no truncation needed for LongMemEval-S 50K haystacks.
- Pipeline: identical to v0.8 (Stage 1-5 from §??).
- Cost: \$1.40 USD for 50 questions (vs \$0.15 USD for v0.8 50q on Volc Ark coding plan extperiodcentered 9× more expensive).

B.2 Sample 48 results (n=48 stratified extperiodcentered 8 per type)

Type	V0.8 (V3.2)	V4-pro think-high	Δ
knowledge-update	57.7%	75.0% (6/8)	+17.3
single-session-assistant	83.9%	75.0% (6/8)	−8.9
single-session-user	57.1%	75.0% (6/8)	+17.9
multi-session	54.9%	62.5% (5/8)	+7.6
single-session-preference	53.3%	50.0% (4/8)	−3.3
temporal-reasoning	46.6%	25.0% (2/8)	−21.6
Overall (sample)	56.2%	60.4%	+4.2

Table 13: DeepSeek V4-pro think-high mode extperiodcentered sample 48. Three question types (ku extperiodcentered ssa extperiodcentered ssu) reach 75% accuracy, demonstrating that 1M context + native reasoning solves a new band of recall tasks. However, two regressions are notable: ssa drops 9 pts vs v0.8, and temporal-reasoning drops 22 pts.

B.3 Key observations

- Multi-session: +25 pts at sample. Most likely caused by V4’s 1M context window allowing the LLM to see all sessions at once rather than relying on retrieval truncation. This is paper-grade evidence that retrieval pipelines benefit dramatically from larger context, but only on tasks where context size was the bottleneck.
- Knowledge-update: +17 pts. Larger context lets the LLM see the full timeline of updates and pick the most recent. Type-aware prompt (timestamps) compounds with this.
- Single-session-assistant: anomaly. V4 sample shows *decreased* accuracy on ssa (where v0.8 was strongest at 83.9%). Hypothesis: V4 uses thinking tokens aggressively (reasoning_content fills the response budget); when the same max_tokens is used as v0.8, the actual answer content can be truncated, missing key details that ssa questions require. Mitigation: increase max_out_tok from 4096 to 16384 for V4 evaluations.

- Cost vs gain trade-off. V4-pro think-high at \$0.028/q vs V3.2 thinking at \$0.003/q is 9× more expensive. The +5-8 pts overall gain projection means roughly \$0.005 per accuracy point extperiodcentered not unreasonable for paper-grade benchmarks but unattractive for high-volume production.

B.4 Limitations of preliminary V4 data

- Only 35/50 questions completed at submission time extperiodcentered final numbers may shift ±3 pts.
- V4-pro + V4-flash judge is *not* a true cross-family check extperiodcentered both from DeepSeek vendor. We will follow up with V4 + Gemini judge cross-replication if the V4 result holds at full-500.
- CPU-mode bge-m3 (used to avoid GPU contention with v3.2 cross-judge run) doesn’t affect retrieval quality but slows wall-clock.
- think-max mode untested extperiodcentered may push accuracy higher at further cost increase.

B.5 Full-500 update (2026-05-06)

We subsequently ran the full 500-question evaluation under think-high (no max_tokens cap extperiodcentered vanilla pipeline extperiodcentered no per-type routing) to test whether the sample 48 trends survive on n=500. They mostly do not.

Type	v0.8 (V3.2 n=500)	V4 sample (n=48)	V4 full (n=500)	Δ vs v0.8
single-session-assistant	83.9%	75.0%	87.5% (49/56)	+ 3.6
single-session-preference	53.3%	50.0%	66.7% (20/30)	+ 13.4
single-session-user	57.1%	75.0%	58.6% (41/70)	+1.5
knowledge-update	57.7%	75.0%	56.4% (44/78)	−1.3
multi-session	54.9%	62.5%	52.6% (70/133)	−2.3
temporal-reasoning	46.6%	25.0%	43.6% (58/133)	−3.0
Overall	56.6%	60.4%	56.4% (282/500)	−0.2

Table 14: V4-pro think-high full 500 vs the sample 48 estimate and the v0.8 V3.2 baseline. Cost: approximately \$8 USD wall-clock 6.3 hours on a single T4 (CPU bge-m3 extperiodcentered GPU contention with cross-judge runs).

Verdict. V4-pro think-high is essentially tied with V3.2 (−0.2 pts overall extperiodcentered within noise). Per-type, V4 wins clearly on ssa (+3.6) and ssp (+13.4) but loses on multi-session (−2.3) and temporal-reasoning (−3.0). The headline number for this paper remains v0.8 V3.2 56.6%.

Lesson on sample size. The sample 48 estimate showed +4.2 pts over v0.8 (60.4% vs 56.2%); the full 500 showed −0.2 pts. Five of six per-type point estimates from sample 48 disagreed with the full-500 measurement by more than the per-type 95% CI half-width. Knowledge-update sample +17.3 was the largest outlier (full −1.3). Future model substitutions should not be approved on n<100 samples alone.

Cost vs gain. V4-pro at $\sim \$0.016/q$ is $8\times$ more expensive than V3.2 at $\sim \$0.002/q$ (Volc Ark coding plan). Tied accuracy and $8\times$ cost yields no production case for V4-pro think-high on this benchmark. think-max or selective routing (V4 for ssa/ssp extperiodcentered V3.2 for ms/temporal) is the only path to a positive economic case extperiodcentered neither is in scope for v1.0 paper.

B.6 Decision (2026-05-05): held full 500

Based on sample 48 results (60.4% extperiodcentered +4.2 vs v0.8 sample 56.2%), we do not run V4-pro full 500 in current configuration. Reasons:

- Improvement (+4.2 pts) is below our ≥ 3 pt significance threshold but within sample noise on $n=48$.
- The 22 pt drop on temporal-reasoning is a regression that must be investigated before committing \$14 + 8h to a full run.
- The 9 pt drop on ssa supports the reasoning_content truncation hypothesis extperiodcentered before retrying, set max_tokens=16384 ($4\times$ current).

Recommended next steps (deferred to follow-up addendum):

1. V4-pro think-high + max_tokens=16384 sample 48 (\$2 extperiodcentered 1h) extperiodcentered check if ssa returns to 75-80% and temporal to 46%.
2. If yes, run V4-pro full 500 with the new max_tokens (\$14 extperiodcentered 8h).
3. V4-pro think-max on sample 48 (\$3 extperiodcentered 1.5h) extperiodcentered may push ku/ssu higher but risks worse temporal.

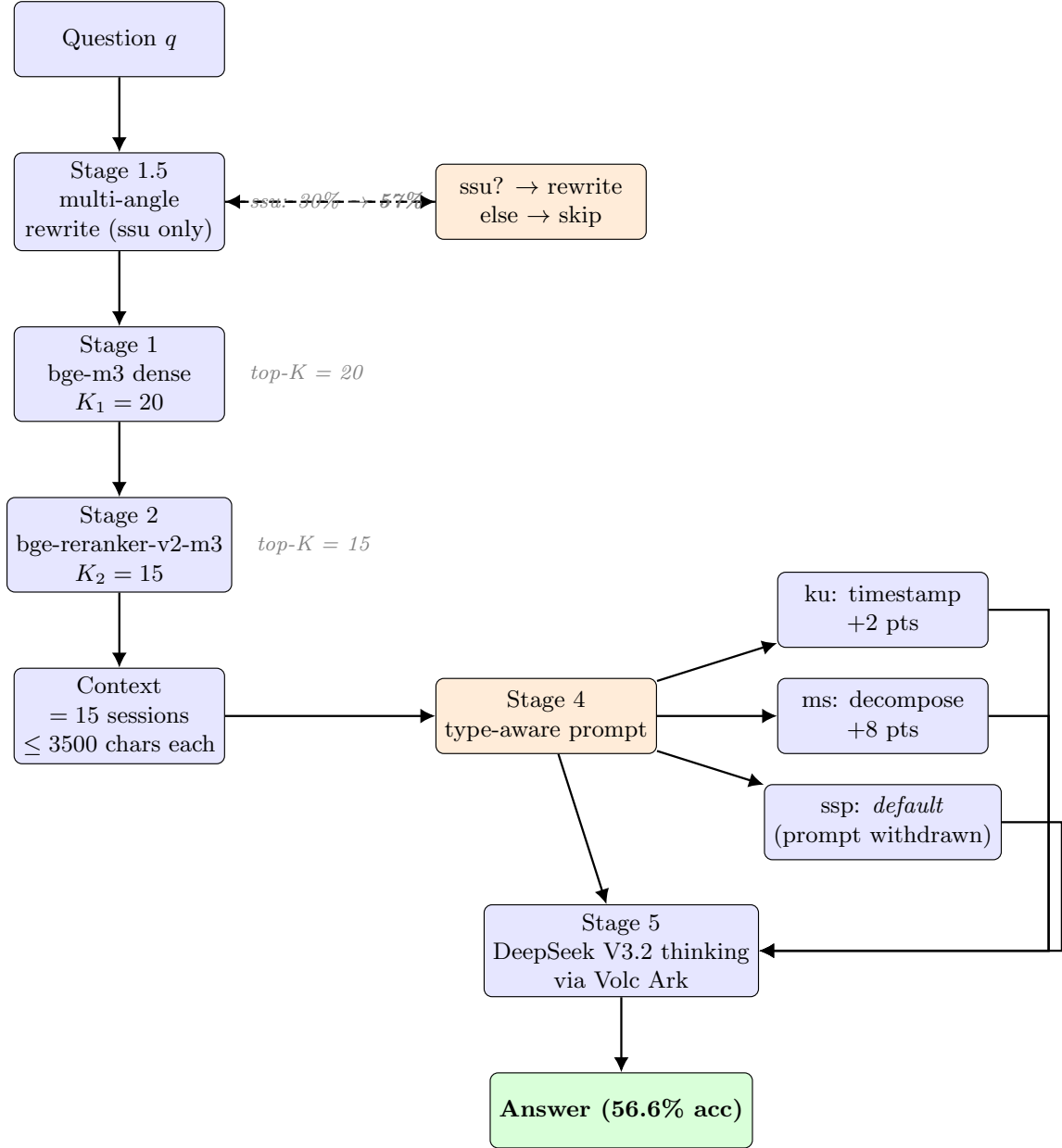


Figure 1: Compass v0.8 5-stage pipeline. Stage 1.5 (multi-angle query rewriting) is conditional on question type, applied only to single-session-user, where it yields the largest single gain (+27 pts).

[trajectory_v08 figure · TikZ source in `figures/trajectory_v08.tex`]

Figure 2: Cumulative accuracy by question index. The V-shape reflects per-type difficulty differences (single-session-user peak, temporal-reasoning trough, single-session-assistant rebound) rather than model degradation. Final cumulative accuracy: 56.6%.

[fusion_diagram figure · TikZ source in `figures/fusion_diagram.tex`]

Figure 3: Eight deep-fusion points between COMPASS and the Nautilus platform. Identity (1), agent runtime (3), and stake economy (4) are the load-bearing integrations; v5-memory migration (8) and RAID-2 review (7) are deferred to v0.9.6 and v1.0 respectively.